Route Wise — Product Requirements Document (PRD)

DOCUMENT: PRD-v2.0
TEAM: Cyber Knight
AUDIENCE: Engineers & Mentors
STATUS: Production Ready

Product Specifications, ML Forecasting Engine & Core Features — Explaining How It Works in Simple English

Product Overview & Problem Solved: Commuters waste 30 to 45 minutes at congested junction bottlenecks without knowing which nearby route is less crowded. **Route Wise** is an intelligent, traffic-aware advisory platform with a Node.js REST API and modern dark glass React/Leaflet dashboard. It fetches driving alternatives via live TomTom telemetry, ranks them using customizable multi-criteria penalty scoring, predicts future slowdowns using an official public **ARIMA (1,0,1)** ML model, and generates transparent side-by-side trade-off explanations so commuters save time and fuel.

1. How It Works in Real-Time: Step-by-Step Flow

To easily explain how the system works to mentors and judges, here is the exact 5-step journey of a single user request:

Step	System Action	Real-Time Example (Trichy: Central Bus Stand to Chatram / Coimbatore)
Step 1 Input	**Geocodes Locations**	User enters origin 'Tiruchirappalli' and destination 'Coimbatore'. Geocoding service converts them to GPS coordinates via TomTom (with instant local coordinate fallback).
Step 2 Ingest	**Fetches Up to 3 Paths**	Queries live TomTom Routing API for all available alternatives. Ingests distance (km), free-flow baseline (s), and real-time live junction congestion delay (s).
Step 3 Forecast	**ARIMA ML Prediction**	Feeds 12-step rolling velocity series into open-source **ARIMA (1,0,1)** model from GitHub to project speed changes and delay spikes at +15m, +30m, and +45m horizons.
Step 4 Score	**Multi-Criteria Ranking**	Applies composite weighting: Score = (50% × Time) + (35% × Delay) + (15% × Distance). Heavily penalizes junction gridlock and ranks Route #1 as top pick.
Step 5 Display	**Explainable Dashboard**	Renders pitch-black glass cards with A/B indicator rows, colored Leaflet polylines, 15-min forecast HUD, and a side-by-side comparison matrix with 60s auto-refresh timer.

2. Core Functional Requirements (What We Built)

Feature ID	Module Name	What It Does (Simple English)	Status
FR-1	**Smart Geocoder**	Converts plain text city/junction names to coordinates. If internet or API fails, instantly switches to offline dictionary so demo never crashes.	**Complete**
FR-2	**Multi-Route Fetcher**	Requests candidate driving routes from TomTom Routing API with real-time live traffic delay metrics and polyline geometry.	**Complete**
FR-3	**ARIMA Forecasting**	Integrates public open-source zemlyansky/arima WebAssembly ML model from GitHub to project speed and transit delay over 15m, 30m, and 45m horizons.	**Complete**
FR-4	**Weighted Scoring Engine**	Ranks routes mathematically based on Travel Time (50%), Delay Penalty (35%), and Distance (15%). Route with highest score (0-100) is #1 pick.	**Complete**
FR-5	**Transparent Rationale**	Generates plain-English comparison bullets (e.g. 'Saves 14m travel time & 37.1 km shorter vs Alt #2') and a full side-by-side comparison matrix table.	**Complete**
FR-6	**API Key Security**	Requires X-API-Key header. Keys are cryptographically hashed using SHA-256 with rate-limiting (100 req/min) and developer status endpoint.	**Complete**
FR-7	**Glassmorphic Dashboard**	Pitch-black glass interface with circular A/B indicator rows, quick presets (Trichy→Coimbatore, Salem→Bangalore), and interactive Leaflet map.	**Complete**
FR-8	**Live Auto-Refresh (60s)**	Visual 60-second countdown timer that automatically refreshes delay metrics and re-evaluates ranking without reloading the page.	**Complete**

3. Technical Architecture, APIs, NFRs & Roadmap

3.1 System Architecture: Clean 3-Tier Design

The application is built on a modular decoupled architecture: **Frontend:** React 18 + Vite + Leaflet maps + Bootstrap 5 dark glassmorphism. **Backend:** Node.js + Express.js REST API with input validation & SHA-256 authentication. **AI & Traffic Layer:** Public open-source **ARIMA (1,0,1)** ML engine + TomTom Traffic Flow raster tiles & Routing API with resilient coordinate dictionary fallback.

3.2 REST API Endpoints Specification

Endpoint	Method & Auth	What It Does & Expected Response	Response Time
/api/routes/recommend	POST X-API-Key req	Accepts origin & destination. Returns ranked routes with GeoJSON coordinates, duration, traffic delay, ARIMA 15m forecast, and comparative trade-off rationale.	< 350ms (Live) < 30ms (Fallback)
/api/routes/refresh	POST X-API-Key req	Refreshes live traffic delay and re-computes ranking scores for existing routes without re-geocoding locations.	< 250ms (Live) < 20ms (Fallback)
/api/health	GET Public (No auth)	Returns system status, TomTom connectivity state, ARIMA model readiness, and server uptime.	< 15ms
/api/developer/status	GET X-API-Key req	Validates API key and returns remaining quota and rate-limit status.	< 20ms

3.3 Performance, Security & Automated Test Coverage

Metric / Requirement	Target Specification	How We Achieved It
API Speed (Latency)	< 450 ms response time	Streamlined native fetch calls, light JSON payloads, and local coordinate database lookup.
Zero-Crash Reliability	100% demo availability	Resilient network timeout handling and built-in coordinate fallback ensure routes display even if external API lags.
API Security	SHA-256 hashed keys	Keys are never stored in plain text; Express rate-limiter prevents spam or denial of service.
Automated Test Suite	40 unit & integration tests	Installed official TDD skill from GitHub (addyosmani/agent-skills). 11 client tests + 29 backend tests passing at 100%.

3.4 Product Roadmap (Now vs Next)

Phase	Timeline	Key Features & Deliverables
Phase 1: Hackathon MVP	Current (Q4 2026)	Core 3-route ranking engine, Leaflet map, ARIMA 15m traffic forecast, comparative matrix, dark glass cards, 40 automated tests.
Phase 2: Multi-Stop Fleet	Q1 2027	Multi-stop traveling salesperson delivery sequencing, automated webhook alerts on unexpected congestion spikes, Python SDK.
Phase 3: Smart EV Fleet	Q2 2027	Electric Vehicle (EV) battery consumption optimization, hill-incline factor, weather condition overlays, enterprise fleet portal.

3.5 2-Minute Live Demo Script for Judges & Mentors:
Step 1 (Open Dashboard): Show the clean pitch-black dark glass dashboard, Leaflet map, and 'AI Engine Live' badge.
Step 2 (Click Preset): Click 'Trichy → Coimbatore' preset pill. Origin and destination populate and routes calculate instantly.
Step 3 (Explain the Ranking & AI Trade-offs): Show how Route #1 is recommended because it saves 14 mins and bypasses congested junctions. Point out the Side-by-Side Comparison Matrix.
Step 4 (Show 15-Min ARIMA Forecast & TDD): Point to the ARIMA (1,0,1) forecast HUD predicting speed trends (+15m, +30m, +45m) and mention our 40 automated tests passing at 100%.